NirBug

An AI-Assisted Software Bug Tracking and Issue Management System

Software Engineering – Group Project Report

Member	Name	Roll Number	Primary Responsibility
Member 1	Anamika Chauhan	IIT2025151	Requirements, UML modelling, AI module, Front-end
Member 2	Shreya Kumbar	IIT2025178	Architecture, Database, Back-end services, Security, Testing

Technology: Python 3.11 · Flask · SQLite · scikit-learn · HTML/JavaScript · Tailwind CSS · Chart.js · pytest

Academic Year 2026

Table of Contents

1. Introduction

2. Software Development Life Cycle (SDLC) – Model, Phases & Sprints

3. Software Requirements Specification (SRS)

4. Requirement Analysis & Use Cases

5. Software Modelling (UML)

6. Software Architecture & Design

7. Database Design

8. Implementation

9. AI / Machine-Learning Module

10. Security Design

11. Software Testing

12. Software Quality Assurance

13. Project Planning & Management

14. Risk Management

15. Software Maintenance

16. Conclusion & Future Scope

Appendix A – How to Run

Appendix B – REST API Reference

1. Introduction

1.1 Problem Statement

Software teams discover hundreds of defects during development and after release. When bugs are reported through e-mail, chat or spreadsheets they get lost, duplicated, assigned to the wrong person or forgotten. Managers cannot see which bugs block a release, developers are overloaded unevenly, and nobody knows the history of who changed what. This causes delayed releases, poor software quality and unhappy users.

1.2 Proposed Solution

NirBug is a web-based bug tracking and issue management system that provides one central place to report, classify, assign, fix, test and close software bugs. It enforces a well-defined bug life cycle, uses machine learning to predict bug severity and detect duplicate reports, automatically assigns bugs to the most suitable developer, tracks versions and releases, and keeps a tamper-evident audit log.

1.3 Objectives

- Provide structured bug reporting with severity classification and priority derivation.
- Enforce a role-controlled bug life cycle (state machine).
- Assign developers manually or automatically based on skills and workload.
- Track version found / target release and compute release readiness.
- Track testing status (FIXED → IN_TESTING → VERIFIED / REOPENED).
- Use AI to reduce triage effort (severity prediction, duplicate detection).
- Apply software engineering practices across the full SDLC.

1.4 Scope

In scope: multi-project bug tracking, five user roles, AI triage, assignment, comments, history, notifications, releases, dashboard analytics, audit log, REST API and a responsive web UI. Out of scope (future work): e-mail delivery, Git/CI integration, file attachments and multi-tenant SaaS hosting.

2. Software Development Life Cycle (SDLC)

The SDLC is the structured sequence of phases a software product goes through, from the first idea to maintenance. For NirBug we used the Iterative Incremental model, run in an Agile / Scrum style.

2.1 Model Chosen: Iterative Incremental (Agile / Scrum-based)

The whole system is not built in one go. It is divided into small increments, and each increment is developed in a short time-box called a sprint. Every sprint goes through all the SDLC phases (plan, analyse, design, implement, test, review), so at the end of every sprint there is a working, tested part of the product. The next sprint adds more features on top of it (incremental) and improves what already exists using feedback (iterative).

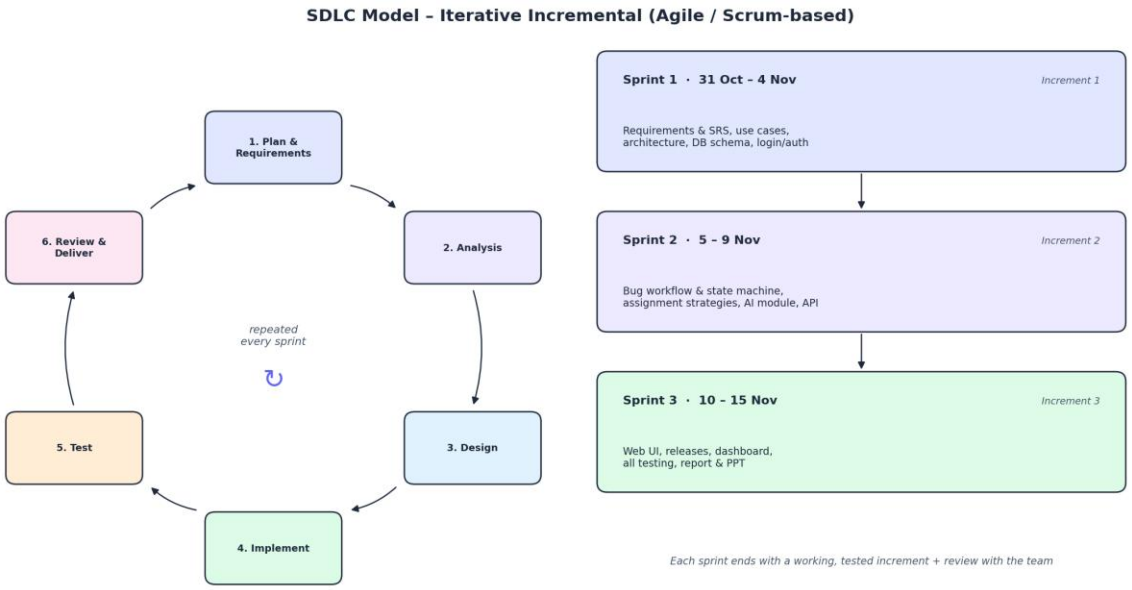


Figure 2.1 – SDLC model: each sprint runs the full cycle and adds one increment

2.2 Phases of the Life Cycle (as applied in NirBug)

Phase	What happens	Output / artefact in this project
1. Planning & Requirements	Understand the problem, identify users and features, fix scope and schedule	Problem statement, SRS (Section 3), Gantt chart, risk table
2. Analysis	Study stakeholders, use cases, feasibility; refine and prioritise requirements	Stakeholder list, use-case diagram & descriptions, feasibility study (Section 4)
3. Design	Decide architecture, modules, database and UI; model the system with UML	Layered architecture, class/sequence/activity/state diagrams, ER diagram, design patterns (Sections 5–7)
4. Implementation (Coding)	Write the code module by module following the design	Flask back-end, SQLite DB, AI module, web front-end (Sections 8–10)
5. Testing	Verify each part and the whole system against the requirements	63 automated unit, integration and system tests, traceability matrix (Sections 11–12)
6. Deployment & Review	Run the increment, demo it, collect feedback, plan the next sprint	Working increment at the end of each sprint, final demo on 15 Nov
7. Maintenance	Fix, adapt and improve the system after delivery	Maintenance plan (Section 15)

2.3 Sprints (Increments) on Our Timeline

Sprint	Dates	Increment delivered	Phases covered
Sprint 1	31 Oct – 4 Nov 2026	SRS, use cases, architecture, DB schema, login & registration	Planning, Analysis, Design, first Implementation + tests
Sprint 2	5 – 9 Nov 2026	Bug life cycle (state machine), assignment strategies, AI module, REST API	Design refinement, Implementation, unit & integration tests
Sprint 3	10 – 15 Nov 2026	Web UI, releases, dashboard, notifications, audit log, report & PPT	Implementation, system testing, review, deployment/demo

Scrum practices we followed (scaled down for a two-person team): a product backlog (the list of functional requirements FR-01 to FR-16), sprint planning at the start of each sprint, short daily check-ins between the two members, and a sprint review/demo at the end of each sprint where the increment was tested and the next sprint was planned.

2.4 Why This Model? (Comparison with Other SDLC Models)

Model	How it works	Suitable for NirBug?
Waterfall	Phases done once, strictly one after another; no going back	No – requirements (e.g. AI features, UI) changed during the project; nothing works until the very end
V-Model	Waterfall with a test phase planned for each development phase	Partly – good testing discipline (we borrowed this idea), but still rigid
Spiral	Iterations driven by risk analysis; suited to large, high-risk projects	No – too heavy for a 16-day student project
Prototype	Build a throw-away prototype to discover requirements	Partly – we used a quick UI prototype for the report-bug flow
Iterative Incremental (Agile/Scrum) ✓	Small time-boxed sprints, each delivers a working increment	Yes – short deadline, small team, changing requirements, early working product, continuous testing

2.5 Advantages Observed

- A working version existed after Sprint 1, so the risk of having nothing to submit was low.
- Requirement changes (e.g. adding duplicate detection and release readiness) were absorbed in the next sprint.
- Testing happened in every sprint, so bugs were found early and cheaply.
- Work was divided clearly between the two members per sprint, making progress easy to track.

3. Software Requirements Specification (SRS)

This section follows the structure of IEEE 830 / ISO/IEC/IEEE 29148.

3.1 Users (Actors) and Characteristics

Role	Description	Key permissions
Reporter	Any end user / client who finds a bug	Register, report bugs, comment, view own bugs
Developer	Fixes bugs; has skills (ui, api, auth …)	Start work, mark FIXED, comment
Tester (QA)	Verifies fixes	Move FIXED → IN_TESTING → VERIFIED / REOPENED, mark DUPLICATE
Manager	Team lead / project manager	Assign bugs, manage projects & releases, view audit log
Admin	System administrator	All manager rights + create users with any role

3.2 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow users to register and log in with username and password.	High
FR-02	The system shall support five roles and restrict actions by role (RBAC).	High
FR-03	A user shall be able to report a bug with project, title, description, component, version found and target release.	High
FR-04	The system shall predict the bug severity (CRITICAL/HIGH/MEDIUM/LOW) using ML when the user does not choose one.	High
FR-05	The system shall derive priority (P1–P4) from severity, raising it if the bug targets a release.	High
FR-06	The system shall show possible duplicate bugs while a bug is being written.	Medium
FR-07	A manager shall assign a bug to a developer manually or automatically (skill + workload strategy).	High
FR-08	The system shall enforce the bug life cycle; illegal transitions shall be rejected.	High
FR-09	Developers shall only change the status of bugs assigned to them.	High
FR-10	Users shall comment on bugs; every field change shall be stored in bug history.	Medium
FR-11	The system shall notify the assignee/reporter when a bug is assigned, commented or changes status.	Medium
FR-12	Managers shall create projects and releases (versions) and ship a release only if no critical/high bug is open.	High
FR-13	Users shall search and filter bugs by text, project, status and severity.	Medium
FR-14	The system shall show a dashboard (counts by status/severity, workload, mean time to close, AI agreement).	Medium
FR-15	The system shall record security-relevant actions in an audit log viewable by managers/admins.	High
FR-16	A Kanban board shall allow changing bug status by drag-and-drop.	Low

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Passwords hashed with PBKDF2-SHA256 (600 000 iterations, salted); never stored in plain text.
NFR-02	Security	Account locked for 15 min after 5 failed logins; login endpoint rate-limited to 10 req/min/IP.
NFR-03	Security	All SQL uses parameterised queries; all output HTML-escaped; CSP & anti-clickjacking headers.
NFR-04	Performance	API responses < 300 ms for 10 000 bugs on a laptop (indexes on status, assignee, project).
NFR-05	Usability	Responsive UI usable on mobile (≥ 360 px) and desktop; dark/light theme; ≤ 3 clicks to report a bug.

NFR-06	Reliability	Foreign-key constraints and CHECK constraints guarantee data integrity; invalid input returns 4xx, never 500.
NFR-07	Maintainability	Layered architecture, design patterns, ≥ 60 automated tests, PEP-8 code.
NFR-08	Portability	Runs on Windows/Linux/macOS with Python 3.10+; no external DB server needed.
NFR-09	Accountability	Every create/assign/status change is written to bug_history and audit_log with timestamp and actor.
NFR-10	Availability	Stateless API (token auth) so it can be scaled horizontally behind a load balancer.

3.4 Constraints and Assumptions

- Development time of 16 days (31 Oct – 15 Nov 2026) and a team of 2 students.
- Free/open-source technologies only (Python, Flask, SQLite, scikit-learn).
- The AI model is trained on a small labelled dataset; humans can always override its prediction.
- Users access the system with a modern browser (Chrome, Edge, Firefox, Safari).

4. Requirement Analysis

4.1 Stakeholders

Stakeholder	Interest / Goal
End users / clients	Report problems easily and know when they are fixed
Developers	Clear, prioritised list of work; fair workload
QA / Testers	Know what to test; reopen failed fixes
Project managers	Visibility of progress, blockers and release readiness
System administrator	Security, user management, audit
Course instructor / evaluator	Demonstration of SE principles

4.2 Requirement Elicitation Techniques

- Study of existing tools (Jira, Bugzilla, GitHub Issues) – document analysis.
- Brainstorming sessions within the team and informal interviews with fellow students who work on projects.
- Prototyping: an early UI mock-up was used to validate the bug-reporting flow.

4.3 Feasibility Study

Type	Assessment
Technical	Feasible – mature open-source stack the team knows (Python, Flask, SQL).
Economic	Feasible – zero licence cost; runs on a laptop or a low-cost cloud VM.
Operational	Feasible – familiar web UI, role-based workflow mirrors real teams.
Schedule	Feasible – scope fixed to 16 FRs across 3 sprints.

4.4 Use-Case Diagram

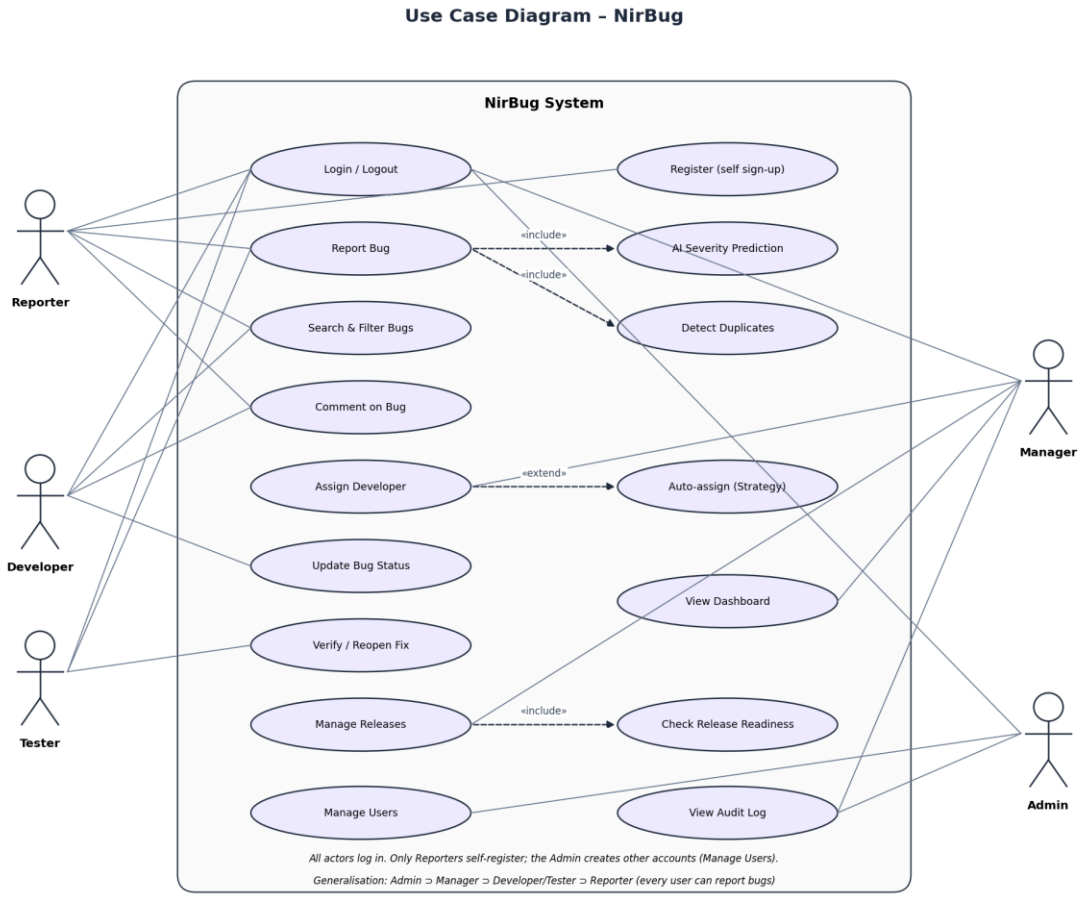


Figure 1 – Use-case diagram

4.5 Use-Case Descriptions

UC-02 Report Bug

Field	Description
Actor	Reporter (any logged-in user)
Pre-condition	User is logged in; at least one project exists
Main flow	1. User opens Report Bug. 2. Enters title & description. 3. System shows AI-predicted severity, component and duplicates. 4. User submits. 5. System validates, derives priority, stores bug with status NEW, writes history & audit, returns bug id.
Alternate flow	3a. Duplicate found → user may cancel and comment on the existing bug. 5a. Validation fails → error message, nothing stored.
Post-condition	Bug exists with status NEW and an AI severity value.

UC-05 Assign Developer

Field	Description
Actor	Manager / Admin
Pre-condition	Bug status NEW or REOPENED
Main flow	1. Manager opens a bug. 2. Chooses a developer or 'Auto'. 3. Auto: SkillBasedStrategy picks the developer whose skills match the component with the lowest weighted workload. 4. Status → ASSIGNED, developer notified.

Exception	Non-manager → 403 Forbidden. No developers → error.

UC-07 Verify / Reopen Fix

Field	Description
Actor	Tester
Pre-condition	Bug status FIXED
Main flow	1. Tester moves bug to IN_TESTING. 2. Tests the fix. 3a. Pass → VERIFIED → CLOSED. 3b. Fail → REOPENED, developer notified.

5. Software Modelling (UML)

The following UML 2.x diagrams model the static structure and dynamic behaviour of NirBug.

5.1 Class Diagram

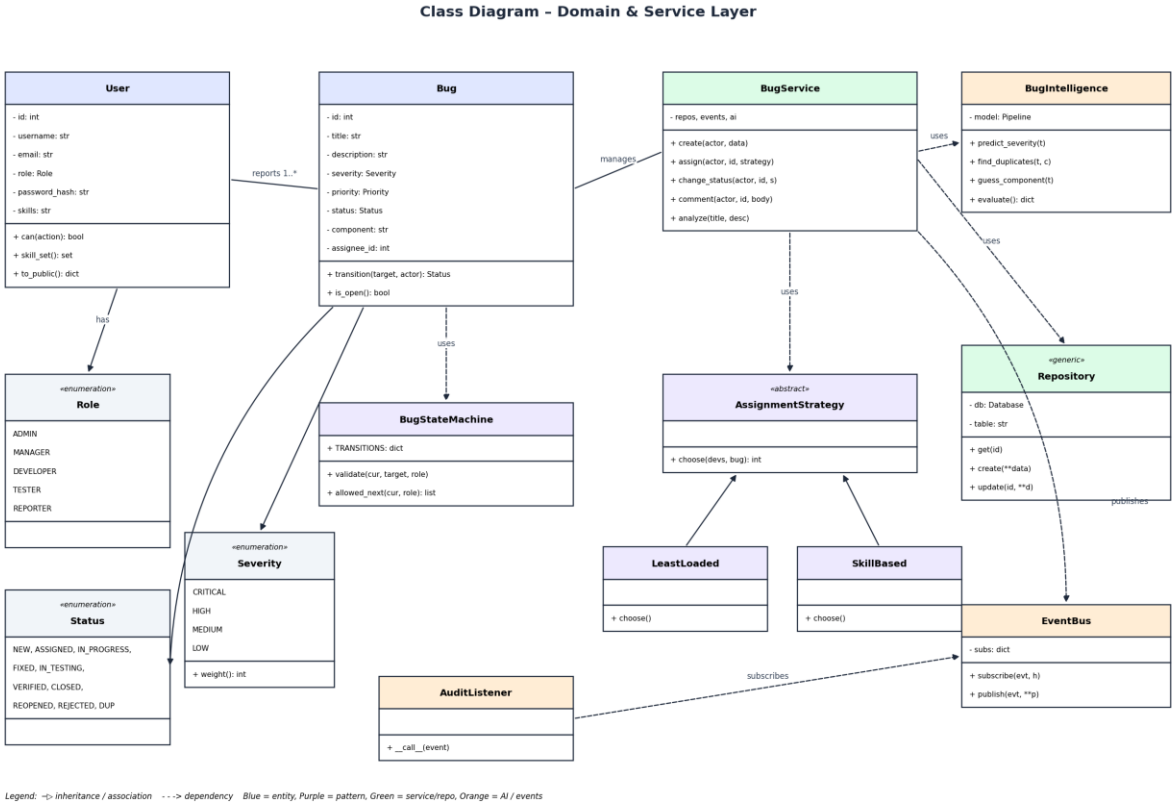


Figure 2 – Class diagram (domain + service layer)

User, Bug and the enumerations form the domain model. BugStateMachine encapsulates life-cycle rules. BugService is the main control class; it depends on the abstract AssignmentStrategy (Strategy pattern), BugIntelligence (AI) and Repository (data access). EventBus with AuditListener implements the Observer pattern.

5.2 Sequence Diagram

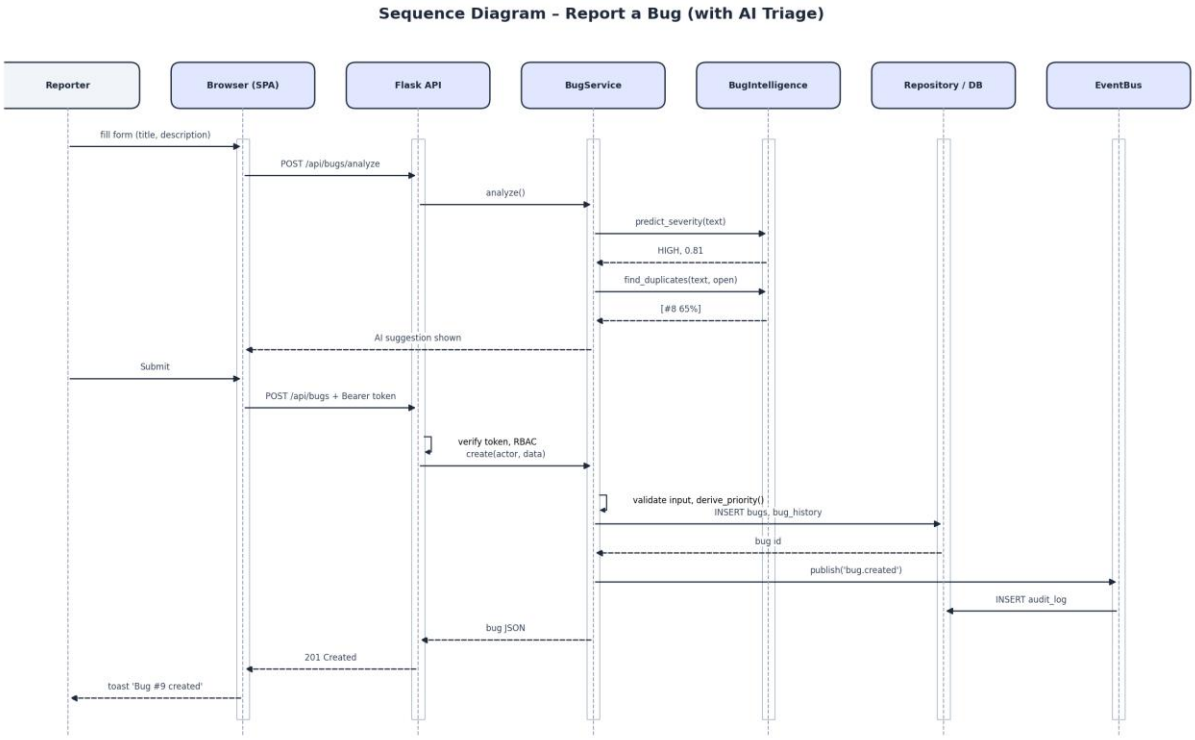


Figure 3 – Sequence diagram for reporting a bug

5.3 Activity Diagram

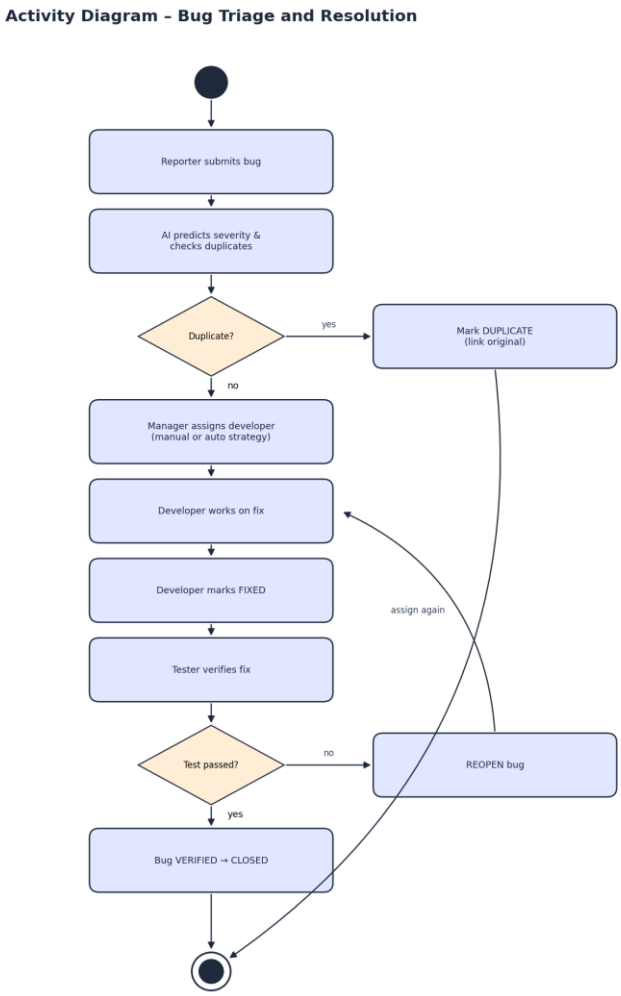


Figure 4 – Activity diagram: triage and resolution

5.4 State Machine Diagram

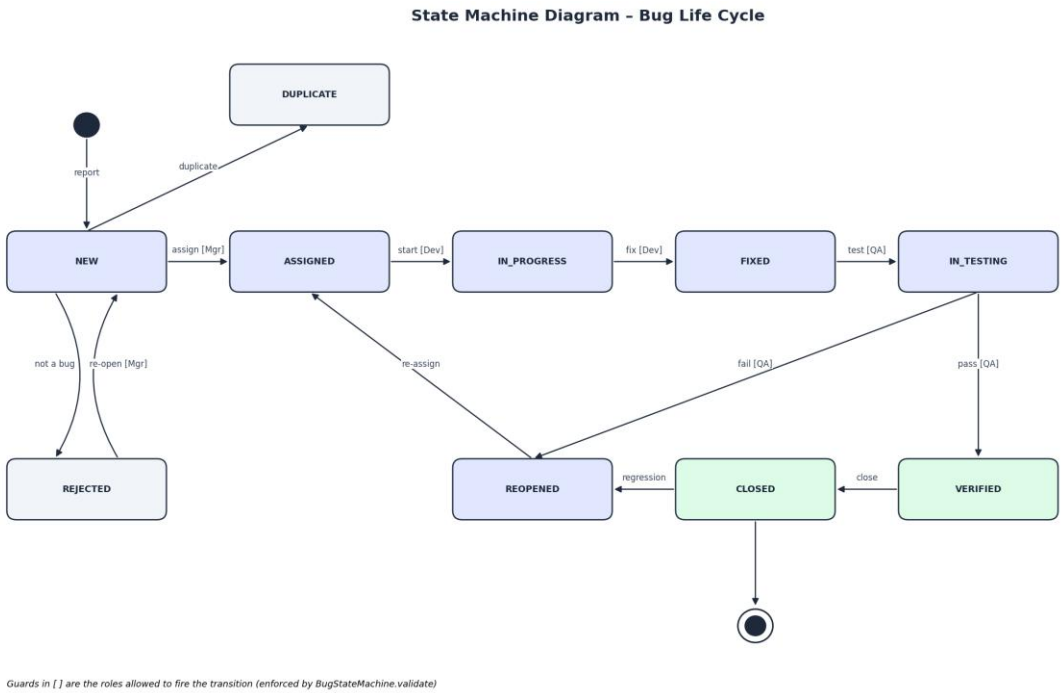


Figure 5 – Bug life-cycle state machine

From	To	Allowed roles
NEW	ASSIGNED / REJECTED / DUPLICATE	Manager, Admin (Tester for DUPLICATE)
ASSIGNED	IN_PROGRESS / REJECTED	Developer (assignee), Manager
IN_PROGRESS	FIXED	Developer
FIXED	IN_TESTING	Tester
IN_TESTING	VERIFIED / REOPENED	Tester
VERIFIED	CLOSED	Tester, Manager
CLOSED	REOPENED	Tester, Manager
REOPENED	ASSIGNED / IN_PROGRESS	Manager / Developer

5.5 Data Flow Diagram

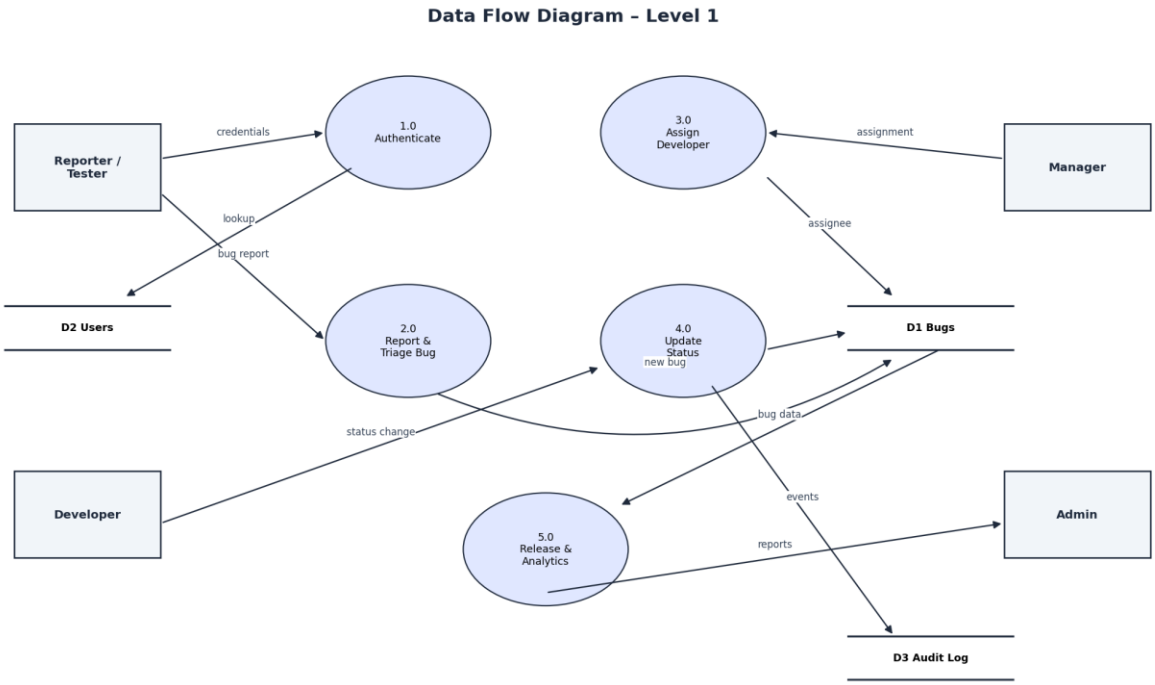


Figure 6 – Level-1 DFD

6. Software Architecture and Design

6.1 Architectural Style – Layered (3-tier, MVC-style)

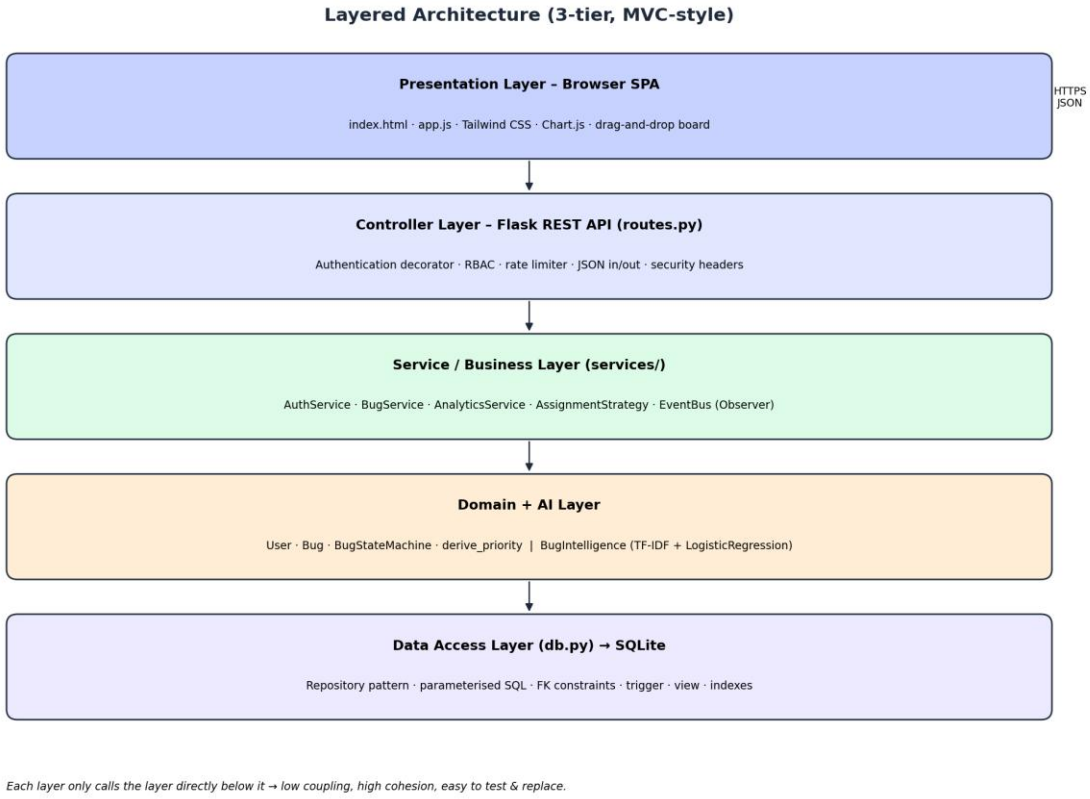


Figure 7 – Layered architecture

Justification: a layered architecture separates presentation, business logic and data access. Each layer can be tested independently (we unit-test the domain without a database), replaced independently (SQLite → PostgreSQL only touches db.py) and understood by new developers quickly. The front-end is a single-page application that talks to a stateless JSON REST API, which also allows a future mobile app to reuse the same back-end.

6.2 Module / Component Responsibilities

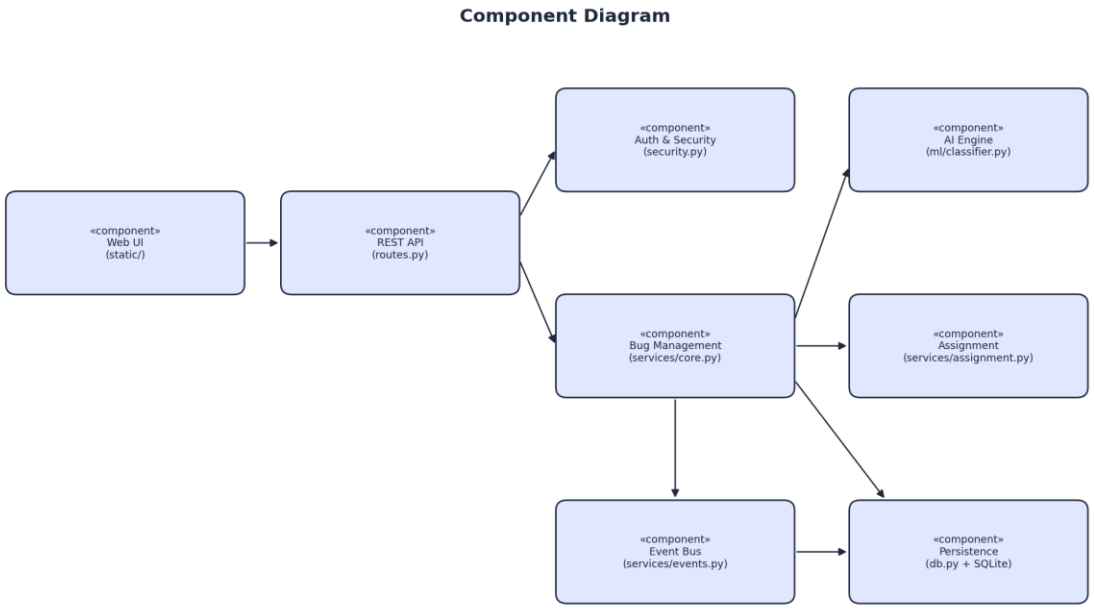


Figure 8 – Component diagram

Module	File	Responsibility
Web UI	static/index.html, app.js, style.css	Screens, forms, charts, Kanban board, XSS-safe rendering
REST API	routes.py	HTTP endpoints, authentication decorator, RBAC check, error mapping
Security	security.py	Hashing, password policy, tokens, rate limiter, security headers
Domain	domain.py	Entities, enums, state machine, priority rule, permission matrix
Services	services/core.py	Use-case logic: auth, bugs, analytics, release readiness
Assignment	services/assignment.py	Strategy pattern – least-loaded & skill-based algorithms
Events	services/events.py	Observer pattern – audit & notification listeners
AI Engine	ml/classifier.py	Severity classifier, duplicate detection, component guess
Persistence	db.py	Schema, repositories, parameterised SQL

6.3 Design Patterns Used

Pattern	Where	Why
Repository	db.py	Hides SQL from business logic; easy to swap database
Strategy	AssignmentStrategy	Interchangeable assignment algorithms (Open/Closed principle)
State (table-driven)	BugStateMachine	Central, testable life-cycle rules
Observer / Pub-Sub	EventBus	Audit log & notifications decoupled from services
Factory	create_app()	Builds configured app instances (prod, test with in-memory DB)
Facade / DI	Services class	Wires dependencies in one place, injects them into services
Decorator	@login_required	Cross-cutting authentication & authorisation

6.4 Design Principles

- **SOLID:** Single responsibility per module; Open/Closed via strategies; Liskov – both strategies substitutable; Interface segregation – small repository classes; Dependency inversion – services depend on abstractions.

- **High cohesion, low coupling:** each layer only calls the layer below it; events remove direct dependencies.
- **DRY:** generic Repository CRUD reused by all tables.
- **Fail-safe defaults:** self-registration can never grant privileged roles; unknown actions are denied.

6.5 Deployment

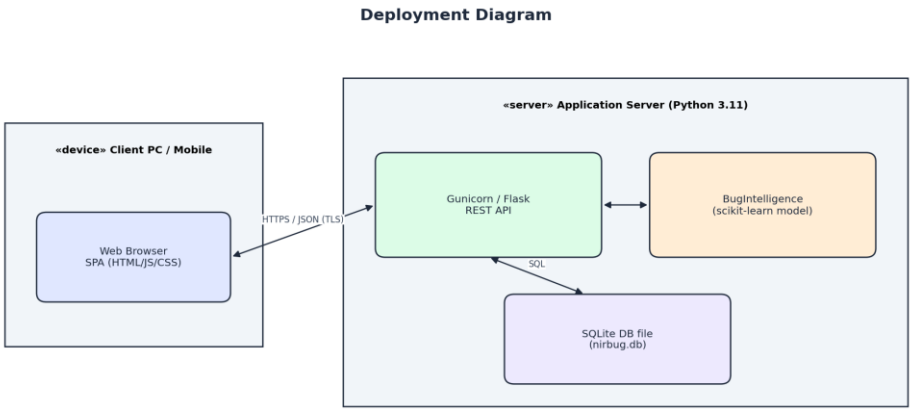


Figure 9 – Deployment diagram

7. Database Design (DBMS)

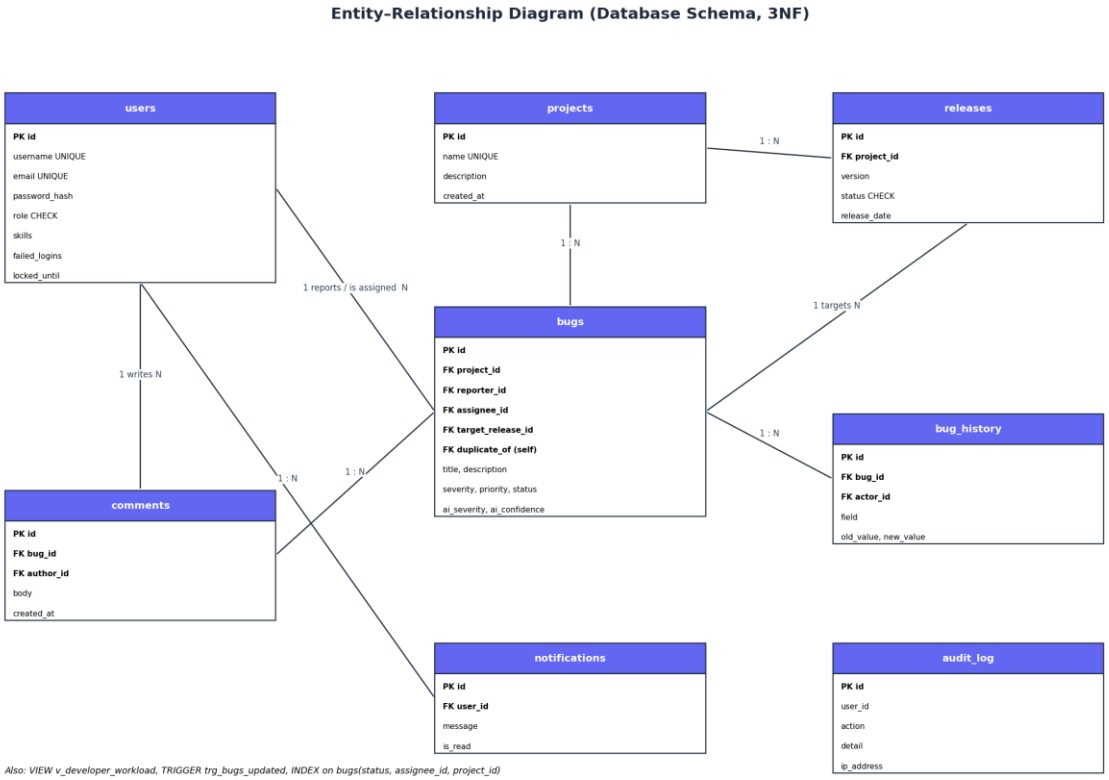


Figure 10 – ER diagram

The schema is in Third Normal Form: every non-key attribute depends only on the primary key (e.g. project name lives only in projects, not repeated in bugs). Relationships are enforced with FOREIGN KEY constraints, and domain rules with CHECK constraints (role, release status) and UNIQUE constraints (username, email, project name, (project, version)).

- **Indexes:** idx_bugs_status, idx_bugs_assignee, idx_bugs_project speed up the most frequent filters.
- **Trigger:** trg_bugs_updated automatically maintains bugs.updated_at.
- **View:** v_developer_workload aggregates open bugs and severity-weighted load per developer (used by auto-assignment).
- **Self-relationship:** bugs.duplicate_of references bugs.id.
- **Transactions:** each write is committed atomically; ON DELETE CASCADE keeps child rows consistent.

8. Implementation

8.1 Technology Stack

Layer	Technology	Reason
Front-end	HTML5, JavaScript (ES2020), Tailwind CSS, Chart.js, Lucide icons	Modern responsive UI without a build step
Back-end	Python 3.11, Flask 3	Lightweight, readable, great for REST APIs
Database	SQLite 3	Zero configuration, ACID, SQL standard
AI / ML	scikit-learn (TF-IDF, Logistic Regression, cosine similarity)	Industry-standard ML library
Security	werkzeug (PBKDF2), itsdangerous (HMAC tokens)	Well-audited primitives
Testing	pytest	Concise tests, fixtures, parametrisation
Docs	matplotlib, python-docx, python-pptx	Diagrams and documents generated from code

8.2 Project Structure

```
bugtracker/
  app.py            application factory, security headers
  routes.py         REST API (controller layer)
  domain.py         entities, enums, state machine, RBAC
  db.py             schema + repositories
  security.py       hashing, tokens, rate limiter
  seed.py           demo data
  services/         core.py, assignment.py, events.py
  ml/               classifier.py, training_data.py
  static/           index.html, app.js, style.css
tests/              test_unit.py, test_integration.py, test_system.py
docs/               diagrams, screenshots, report & PPT generators
```

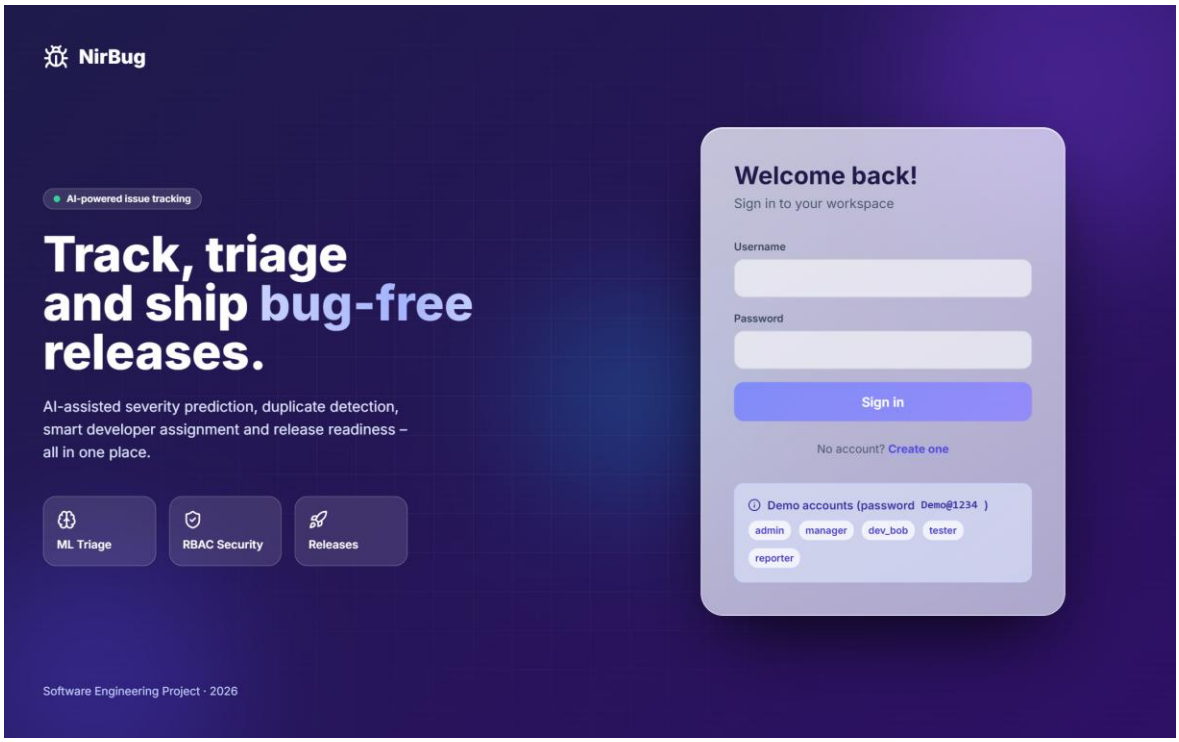
8.3 OOP Concepts Applied

- **Encapsulation:** Bug.transition() is the only way to change a Bug's status; it validates internally.
- **Abstraction:** AssignmentStrategy is an abstract base class (abc.ABC) with abstract method choose().
- **Inheritance:** LeastLoadedStrategy and SkillBasedStrategy inherit AssignmentStrategy; every XxxRepository inherits Repository.
- **Polymorphism:** BugService calls strategy.choose() without knowing which concrete strategy it is.
- **Enumerations & data classes:** Role, Severity, Status, Priority as Enum; User and Bug as @dataclass.

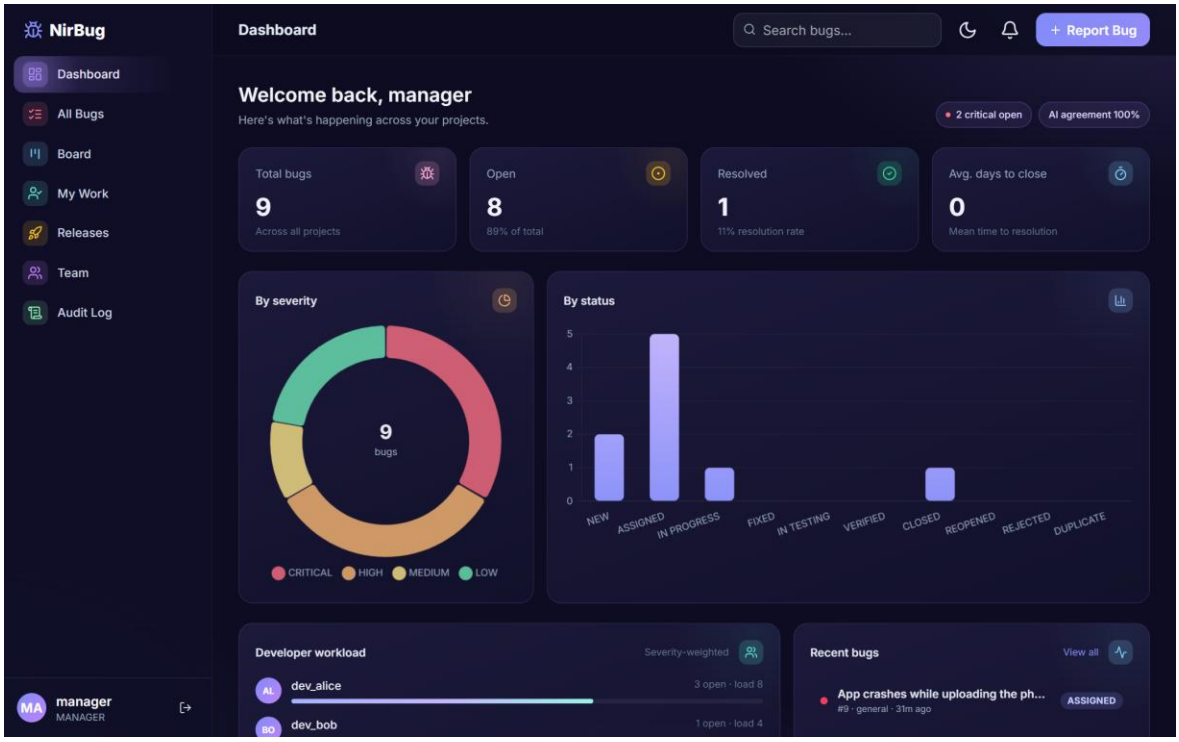
8.4 Key Algorithm – Skill-based Auto-assignment

For every developer d: score(d) = 10 × [bug.component ∈ d.skills] − load(d), where load(d) = Σ severity weight (CRITICAL 4, HIGH 3, MEDIUM 2, LOW 1) of d's open bugs. The developer with the highest score is chosen. This balances expertise and workload. Complexity: O(n) in the number of developers.

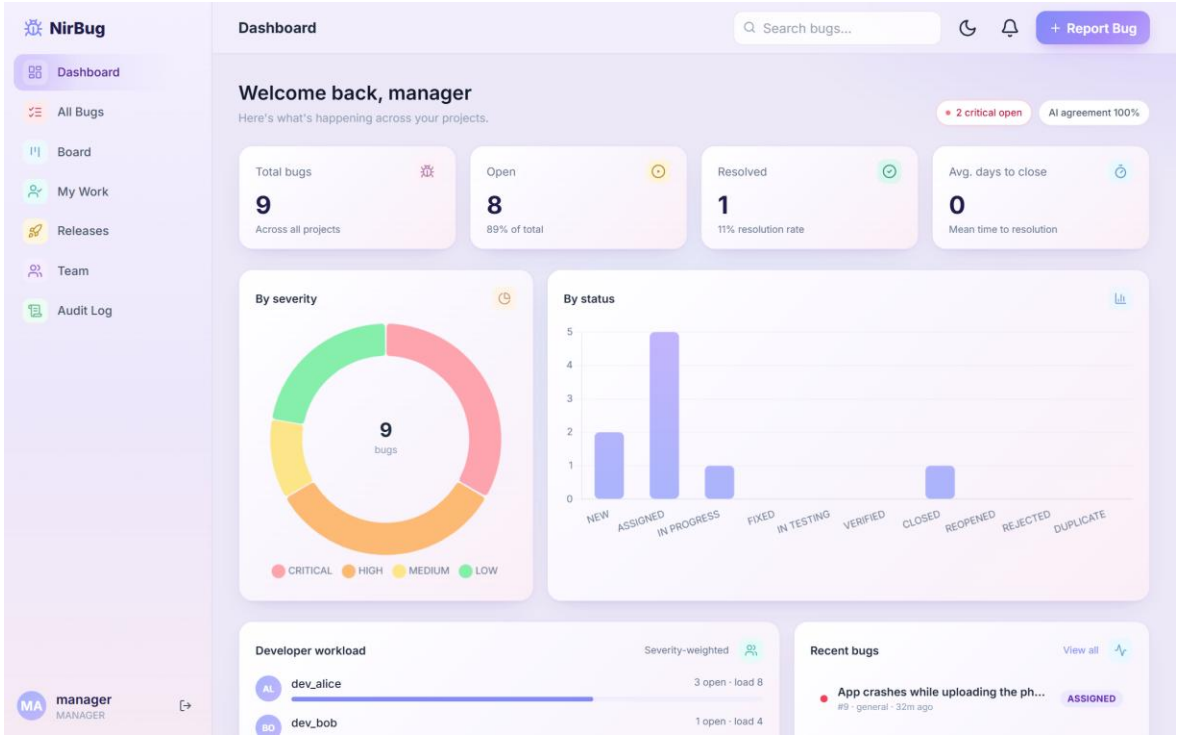
8.5 Screenshots of the System



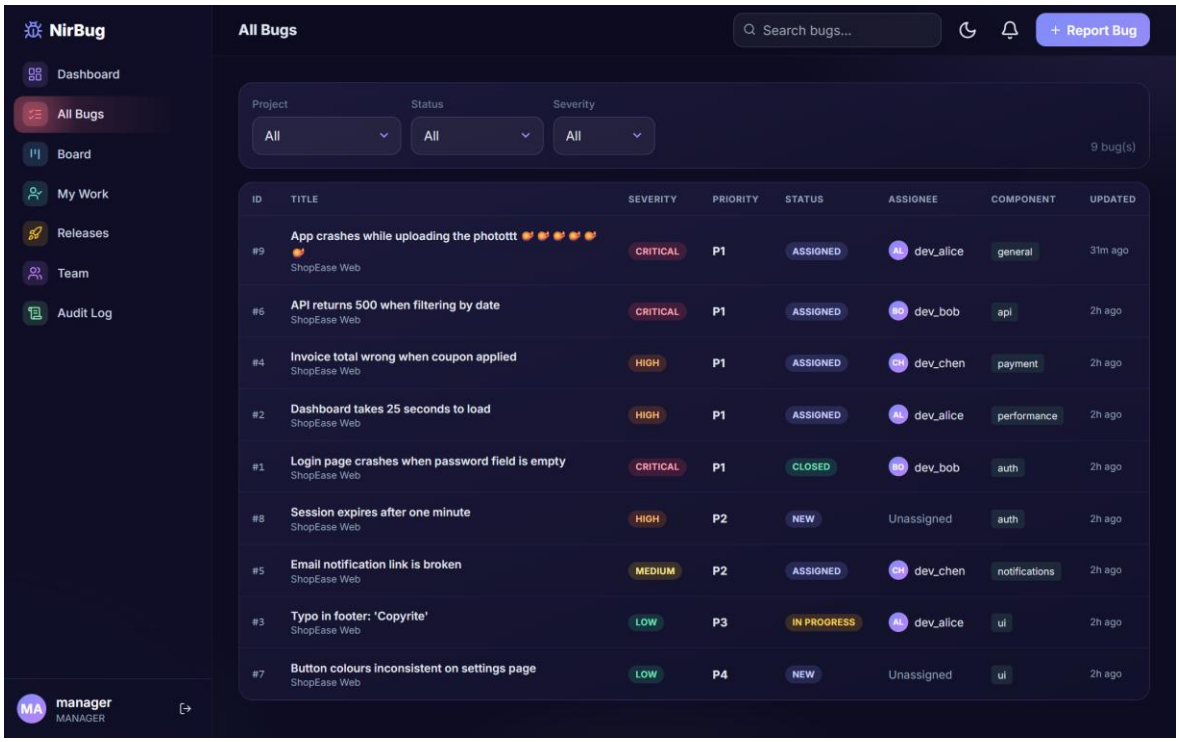
Screenshot – Login screen (animated background, demo accounts)



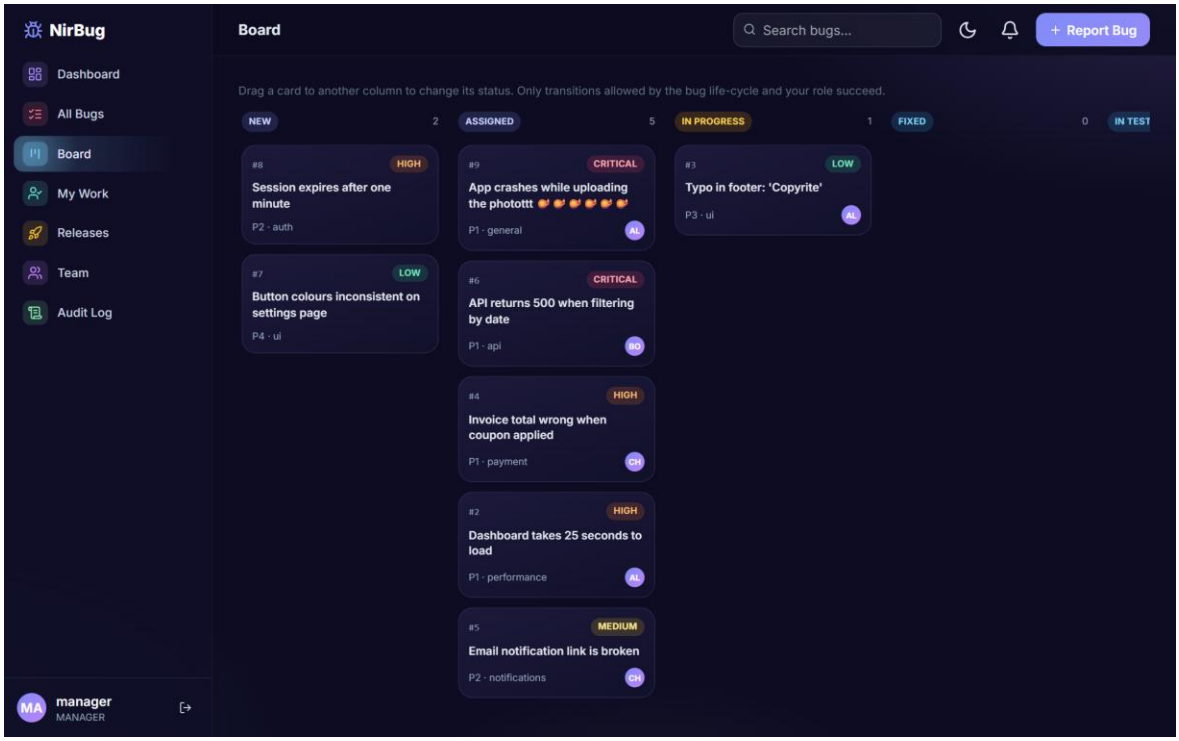
Screenshot – Dashboard – statistics, severity & status charts, workload, recent bugs (dark theme)



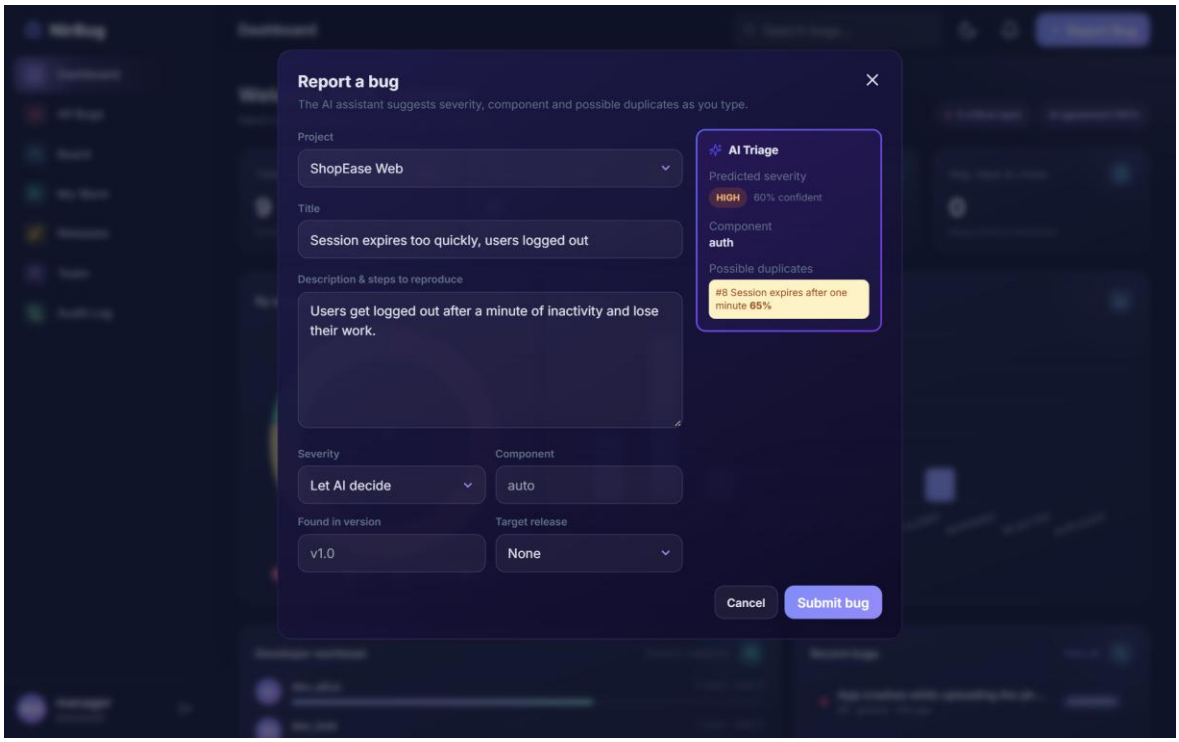
Screenshot – Dashboard in the light theme



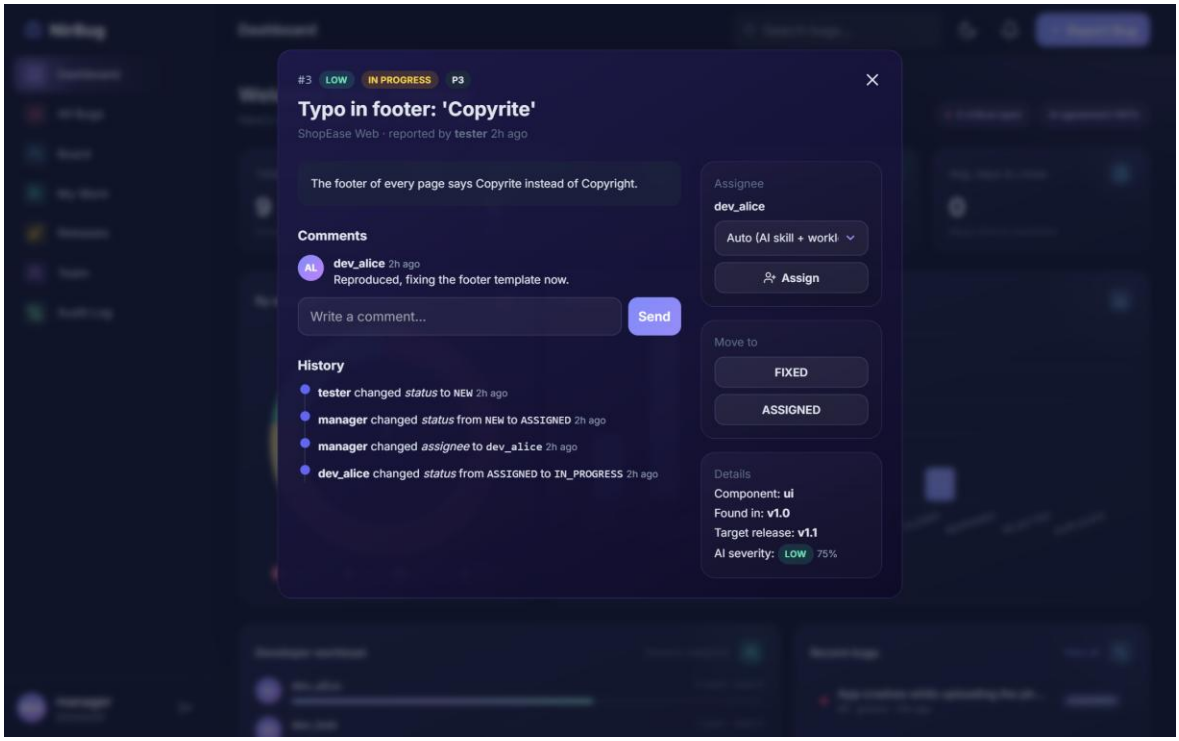
Screenshot – All Bugs – searchable, filterable list with severity, priority and status



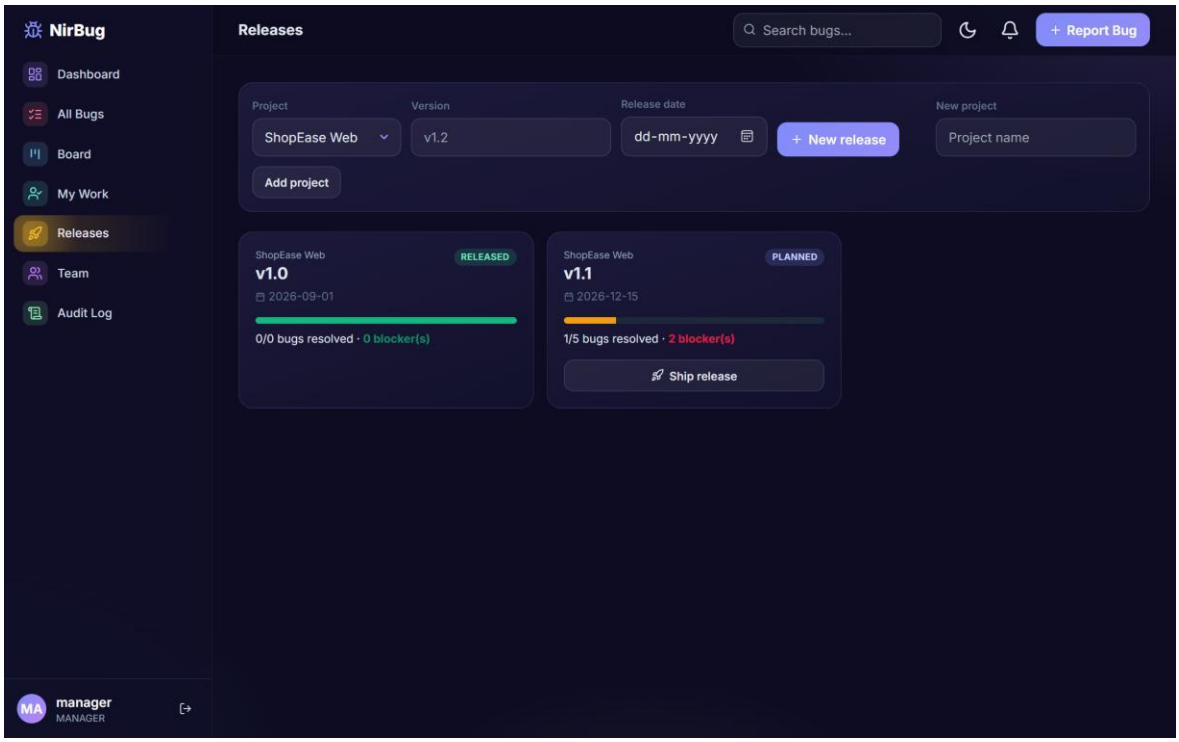
Screenshot – Kanban board – drag a card to change its status (life-cycle rules enforced)



Screenshot – Report a bug – live AI triage: predicted severity, component and possible duplicate



Screenshot – Bug detail – comments, history, assignment and allowed status transitions



Screenshot – Releases – readiness percentage and blockers per version

9. AI / Machine-Learning Module

- **Problem:** manual triage is slow and inconsistent; duplicates waste developer time.
- **Severity classifier:** text = title + description → TF-IDF features (word 1-2 grams + character 2-5 grams) → multinomial Logistic Regression → probability for each class. Output: predicted label + confidence.
- **Hybrid rule layer:** keywords such as 'crash', 'data loss', 'SQL injection' force CRITICAL (safety first).
- **Duplicate detection:** TF-IDF vectors of the new report and all open bugs of the project; cosine similarity ≥ 0.45 is shown as a possible duplicate (top 3).
- **Component guess:** keyword-based expert rules (auth, ui, api, database, payment, performance, notifications).
- **Human-in-the-loop:** the user can override severity; both the AI value and the final value are stored so the dashboard shows the AI/human agreement rate, which is used to monitor model quality in production.

9.1 Evaluation

Metric	Value
Training samples	80
Validation	Stratified 5-fold cross-validation
ML-only accuracy	0.438
Hybrid (rules + ML) accuracy	0.45
Random baseline (4 classes)	0.25

Discussion: with only 80 labelled examples the model is clearly better than random but not perfect, which is why it is used as a suggestion, not a decision. Accuracy is expected to improve as real bugs (with human-corrected labels) are added to the training set – this retraining is part of the maintenance plan (Section 15).

10. Security Design (Cyber-security)

Threat (STRIDE)	Mitigation in NirBug
Spoofing – stolen / guessed passwords	PBKDF2-SHA256 salted hashes, password policy, account lockout, login rate limit
Tampering – forged tokens	HMAC-signed, time-limited tokens (itsdangerous); signature checked on every request
Repudiation	Audit log with user, action, detail, IP and timestamp; bug_history for every field change
Information disclosure	Password hashes never returned by API; generic 'invalid username or password' message
Denial of service	Rate limiter; MAX_CONTENT_LENGTH 1 MB request size
Elevation of privilege	RBAC on every endpoint + in services; self-registration fixed to REPORTER role
SQL injection	Only parameterised queries (?) – verified by test IT-15
Cross-site scripting	Front-end escapes all server data (esc()); Content-Security-Policy header
Clickjacking / MIME sniffing	X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy

10.1 Networking Aspects

The client and server communicate over HTTP using a REST architecture with JSON payloads and standard status codes (200, 201, 400, 401, 403, 404, 409, 423, 429). Authentication is stateless (Bearer token in the Authorization header), so any number of server instances can run behind a load balancer. In production the API must be served over HTTPS (TLS) by a reverse proxy such as Nginx, with the WSGI server Gunicorn.

11. Software Testing

11.1 Test Strategy

We followed the testing pyramid: many fast unit tests, fewer integration tests and a set of black-box system tests through the HTTP API. All 63 automated tests are written with pytest and run with one command (python -m pytest). Techniques used: equivalence partitioning, boundary value analysis (priority rules, password length), state-transition testing (life cycle), negative / security testing.

Level	File	Count	What is tested
Unit	tests/test_unit.py	35	State machine, priority rule, RBAC, hashing, tokens, rate limiter, strategies, AI
Integration	tests/test_integration.py	15	Services + repositories + SQLite: workflow, history, observer, lockout
System	tests/test_system.py	13	Full HTTP API: auth, end-to-end life cycle, RBAC, XSS/SQLi, headers, releases

11.2 Selected Test Cases

ID	Description	Input	Expected	Result
UT-02	Invalid transition	NEW → CLOSED	InvalidTransition raised	Pass
UT-07	Priority boundary	HIGH + blocks release	P1	Pass
UT-10	Password hashing	'Secret123' twice	Different salted hashes; verify OK	Pass
UT-12	Token tampering	Modified token	Rejected (None)	Pass
UT-16	Skill-based assignment	component = auth	Developer with auth skill	Pass
UT-22	Duplicate detection	Similar login bug	Bug #1 returned, unrelated → none	Pass
IT-05	Full life cycle	NEW…CLOSED	7 history entries in order	Pass
IT-06	Developer edits others' bug	dev_bob on chen's bug	403	Pass
IT-11	Brute force	5 wrong passwords	Account locked (423)	Pass
IT-12	Privilege escalation	register role=ADMIN	Role stays REPORTER	Pass
IT-15	SQL injection	' OR 1=1; DROP TABLE bugs	No rows, table intact	Pass
ST-02	No token	GET /api/bugs	401	Pass
ST-05	End-to-end with reopen	9 transitions by 3 roles	CLOSED, ≥ 10 history rows	Pass
ST-06	RBAC	Reporter → /api/audit	403	Pass
ST-10	Rate limit	12 rapid logins	429 returned	Pass
ST-11	Release gate	Ship with open CRITICAL	409, then 200 after resolution	Pass

11.3 Test Results

63 passed, 0 failed (python -m pytest -q). Manual exploratory testing of the UI was also performed on Chrome and Edge in light and dark mode at desktop and mobile widths.

12. Software Quality Assurance

Quality attribute (ISO/IEC 25010)	How it is achieved	How it is measured
Reliability	Validation in services, DB constraints, error handler returning JSON 4xx	Integration/system tests, no 500s in tests
Maintainability	Layered architecture, patterns, small modules, docstrings	Module size, test coverage of core logic
Security	See Section 10	Security test cases (UT-10..14, IT-11/12/15, ST-02/06/08/09/10)
Usability	Consistent UI, colour-coded badges, AI hints, toasts, dark mode, responsive	Task: report a bug in ≤ 3 clicks
Performance	Indexes, SQL view, model trained once at start-up	API timing; analyze() < 50 ms
Functional suitability	Requirements traceability (below)	All FRs covered by tests

12.1 Requirements Traceability Matrix

Requirement	Design element	Test case(s)
FR-01 Register/login	AuthService, TokenManager	ST-03, UT-12
FR-02 RBAC	PERMISSIONS, @login_required	UT-08, ST-06
FR-04 AI severity	BugIntelligence.predict_severity	UT-19..21, UT-24
FR-05 Priority	derive_priority	UT-07
FR-06 Duplicates	find_duplicates	UT-22, ST-07
FR-07 Assignment	AssignmentStrategy	UT-15..18, IT-03/04
FR-08 Life cycle	BugStateMachine	UT-01..06, IT-05/07, ST-05
FR-09 Own bugs only	BugService.change_status	IT-06
FR-11 Notifications	NotificationListener	IT-08, ST-13
FR-12 Releases	release_readiness, ship	IT-10, ST-11
FR-13 Search	BugRepository.search	ST-12, IT-15
FR-15 Audit	AuditListener	IT-08

12.2 Quality Practices

- Coding standard: PEP 8, type hints, docstrings in every module.
- Version control with Git; feature branches and pull-request review between the two members.
- Definition of Done: code + tests passing + reviewed + documented.
- Peer code reviews and a final walkthrough before submission.

13. Project Planning and Management

13.1 Milestones

Milestone	Deliverable	Date
M0	Project assigned, topic chosen	31 Oct 2026
M1	Approved SRS and use cases	2 Nov 2026
M2	Architecture, UML & DB design complete	6 Nov 2026
M3	Feature-complete prototype + tests	12 Nov 2026
M4	Final report, PPT, demo, submission	15 Nov 2026

13.2 Schedule (Gantt Chart)

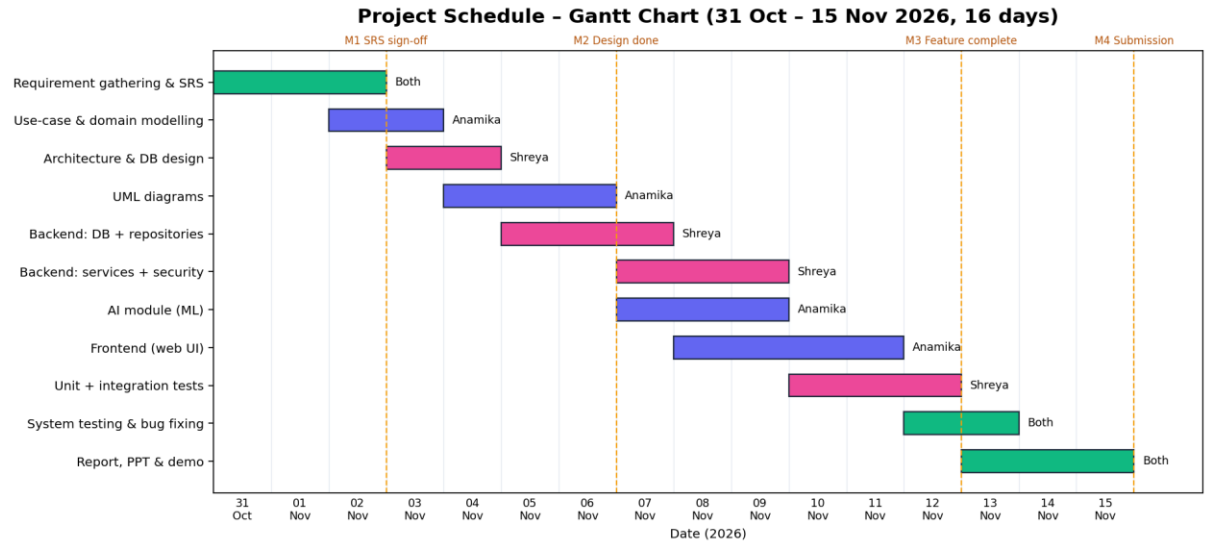


Figure 11 – Gantt chart

13.3 Work Breakdown and Responsibilities

Work package	Anamika Chauhan	Shreya Kumbar
Requirements & SRS	✓ (lead)	✓
UML diagrams	✓	
Architecture & DB design		✓
Back-end (DB, services, API)		✓
Security		✓
AI / ML module	✓	
Front-end UI	✓	
Testing	✓ (system)	✓ (unit/integration)
Report & PPT	✓	✓

13.4 Effort Estimation (COCOMO – Basic, Organic)

Size ≈ 2.2 KLOC (Python + JavaScript). Effort E = 2.4 × (2.2)^1.05 ≈ 5.6 person-months; Development time D = 2.5 × (5.6)^0.38 ≈ 4.8 months for a professional team. The estimate is higher than our 16-day schedule because students worked part-time on a reduced scope, reused open-source libraries heavily and generated documentation from code; the estimate is shown to demonstrate the technique.

14. Risk Management

Risk	Probability	Impact	Mitigation
Scope creep (too many features)	High	High	Fixed FR list; extra ideas moved to Future Scope
Low AI accuracy	Medium	Medium	Hybrid rules, human override, measure agreement
Team member unavailable	Medium	High	Shared Git repo, documented code, both know all modules
Security flaw	Low	High	Security checklist (STRIDE) and dedicated tests
Integration problems between UI and API	Medium	Medium	API contract defined early, system tests

15. Software Maintenance

Type	Example in NirBug
Corrective	Fix defects reported through NirBug itself (dog-fooding); regression tests added for every fix
Adaptive	Move from SQLite to PostgreSQL (only db.py changes); deploy in Docker / cloud
Perfective	Add e-mail/Slack notifications as a new EventBus listener; file attachments; Git commit linking
Preventive	Periodic dependency updates, retraining the ML model, refactoring, increasing test coverage

15.1 How the System Can Be Extended

- New assignment algorithm → create a subclass of AssignmentStrategy and register it in STRATEGIES.
- New notification channel → subscribe a new listener to the EventBus; no service code changes.
- New workflow state → add one entry to BugStateMachine.TRANSITIONS and the Status enum.
- Better AI → add labelled rows (exported from closed bugs with human-corrected severity) and retrain; the evaluate() method reports the new accuracy before deployment.
- Mobile app → reuse the same REST API.

15.2 Maintenance Process

Change requests are themselves logged as issues in NirBug, prioritised by the manager, implemented on a branch, tested by the automated suite (all tests must pass), reviewed, and released with a new version number (semantic versioning MAJOR.MINOR.PATCH). The audit log and bug history support impact analysis.

16. Conclusion and Future Scope

NirBug meets all the required features – bug reporting, severity classification, developer assignment, version tracking, testing status and release management – and adds AI-assisted triage, a Kanban board and analytics. More importantly, it was produced with a systematic SDLC: requirements were specified and traced, the system was modelled with UML, designed with a layered architecture and design patterns, implemented with OOP, secured with defence-in-depth, verified with 63 automated tests, and planned with milestones, a Gantt chart and risk analysis.

Future scope: e-mail/Slack integration, Git and CI/CD integration (auto-close bugs from commit messages), attachments and screenshots, SLA timers, transformer-based NLP models, and multi-tenant cloud deployment.

Appendix A – How to Run

```
pip install -r requirements.txt

python -m bugtracker.app        # open http://127.0.0.1:5000

python -m pytest -v               # run all tests

python docs/generate_diagrams.py  # regenerate diagrams

python docs/build_report.py && python docs/build_ppt.py
```

Demo accounts: admin, manager, dev_alice, dev_bob, dev_chen, tester, reporter – password Demo@1234.

Appendix B – REST API Reference

Method	Endpoint	Role	Purpose
POST	/api/auth/register	public	Create reporter account
POST	/api/auth/login	public	Get token
GET	/api/auth/me	any	Current user
GET/POST	/api/users	any / admin	List / create users
GET/POST	/api/projects	any / manager	List / create projects
POST	/api/projects/{id}/releases	manager	Create release
GET	/api/releases/{id}/readiness	any	Release readiness
POST	/api/releases/{id}/ship	manager	Ship release
GET	/api/bugs?status&severity&project_id&q&mine	any	Search bugs
POST	/api/bugs/analyze	any	AI preview
POST	/api/bugs	any	Report bug
GET/PATCH	/api/bugs/{id}	any / mgr+reporter	Detail / edit
POST	/api/bugs/{id}/assign	manager	Assign (manual/auto)
POST	/api/bugs/{id}/status	by state machine	Change status
POST	/api/bugs/{id}/comments	any	Comment
GET	/api/dashboard	any	Analytics
GET	/api/audit	manager	Audit log
GET	/api/notifications	any	Notifications